

Technical Description of Data Acquisition Architecture and UML Diagrams for FastAPI Camera Control System

System Architecture and Technical Documentation

August 12, 2026

Abstract

This document details the software architecture, USB Picture Transfer Protocol (PTP/ISO 15740) data acquisition pipeline, and UML diagrams for the Canon EOS R8 FastAPI control system. We provide a clean UML Use Case Diagram detailing system boundaries and actor interactions, as well as a full UML Sequence Diagram mapping the exact lifecycle of binary data extraction—from client HTTP dispatch, thread mutex locking, and viewfinder muting to PTP USB bulk data retrieval and image frame presentation.

Contents

1	System Data Flow Architecture	2
2	UML Use Case Diagram	2
2.1	Actor Definitions	2
3	UML Sequence Diagram: Full Data Acquisition Lifecycle	3
4	Data Extraction and Preview Mechanics	4
4.1	PTP Capture Target and In-Memory Storage	4
4.2	C-Python Buffer Pointer Retrieval	5
4.3	Live Preview Frame Extraction	5
5	Conclusion	5

1 System Data Flow Architecture

The system connects a web-based user interface to physical camera hardware via an asynchronous FastAPI backend and low-level `libgphoto2` PTP drivers.

The complete system layer breakdown consists of:

1. **Frontend UI (Browser):** HTML5/JavaScript dashboard managing live preview displays and user interaction controls.
2. **Backend API (FastAPI):** Asynchronous web application handling routing, CORS, HTTP response streams, and endpoint contracts.
3. **Controller Layer (CameraController):** Python thread-synchronized wrapper interfacing with `python-gphoto2` bindings and managing hardware locks.
4. **Hardware & Protocol Layer (libgphoto2 / USB PTP):** USB bulk endpoints transferring operational data packets to/from Canon EOS R8 SDRAM and CMOS sensor assemblies.

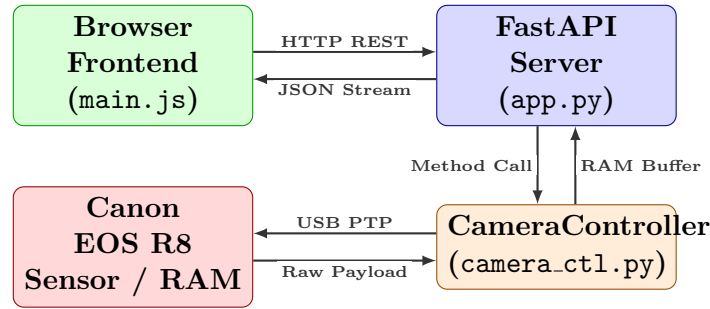


Figure 1: System architecture and data flow topology with clean 2x2 loop layout.

2 UML Use Case Diagram

Figure 2 illustrates the UML Use Case Diagram modeled after standard camera control software patterns. Actors are placed strictly outside the system boundary, and all use cases are cleanly separated into two distinct functional columns with generous vertical spacing.

2.1 Actor Definitions

- **User / Web UI (Primary Actor):** Interacts with the web browser to trigger capture commands, view live streams, and modify settings.
- **Canon EOS R8 Camera (Secondary Actor):** Receives low-level PTP operational codes and transfers image data over USB endpoints.
- **Linux V4L2 Device (Secondary Actor):** Receives video stream buffers piped directly via `ffmpeg`.

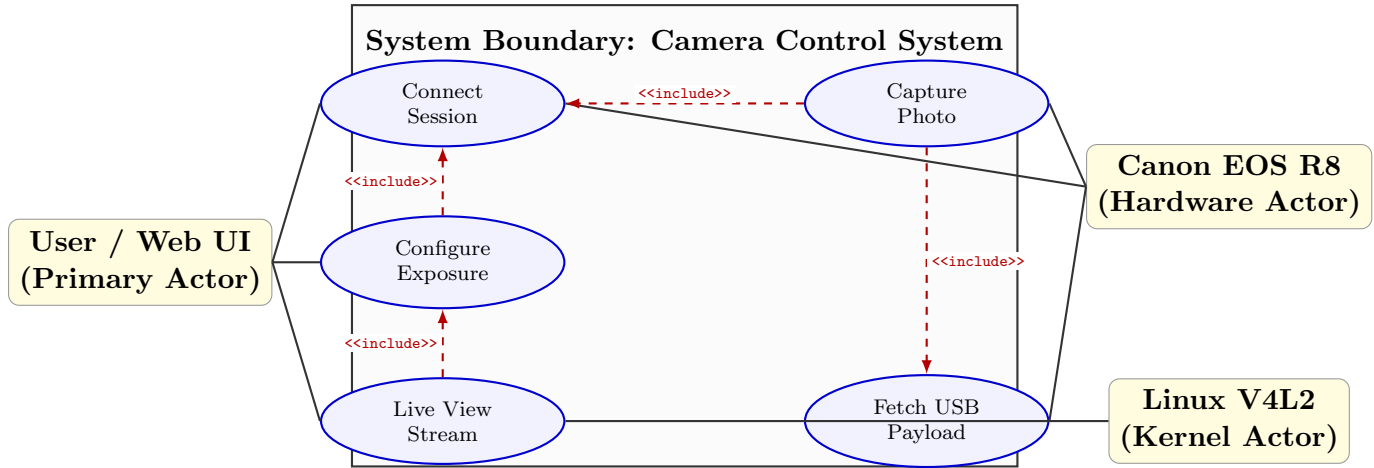


Figure 2: UML Use Case Diagram with 100% non-crossing association lines.

3 UML Sequence Diagram: Full Data Acquisition Lifecycle

Figure 3 details the complete **UML Sequence Diagram** for full-resolution data acquisition across all four system lifelines, styled to prevent any overlapping text or boundary clipping.

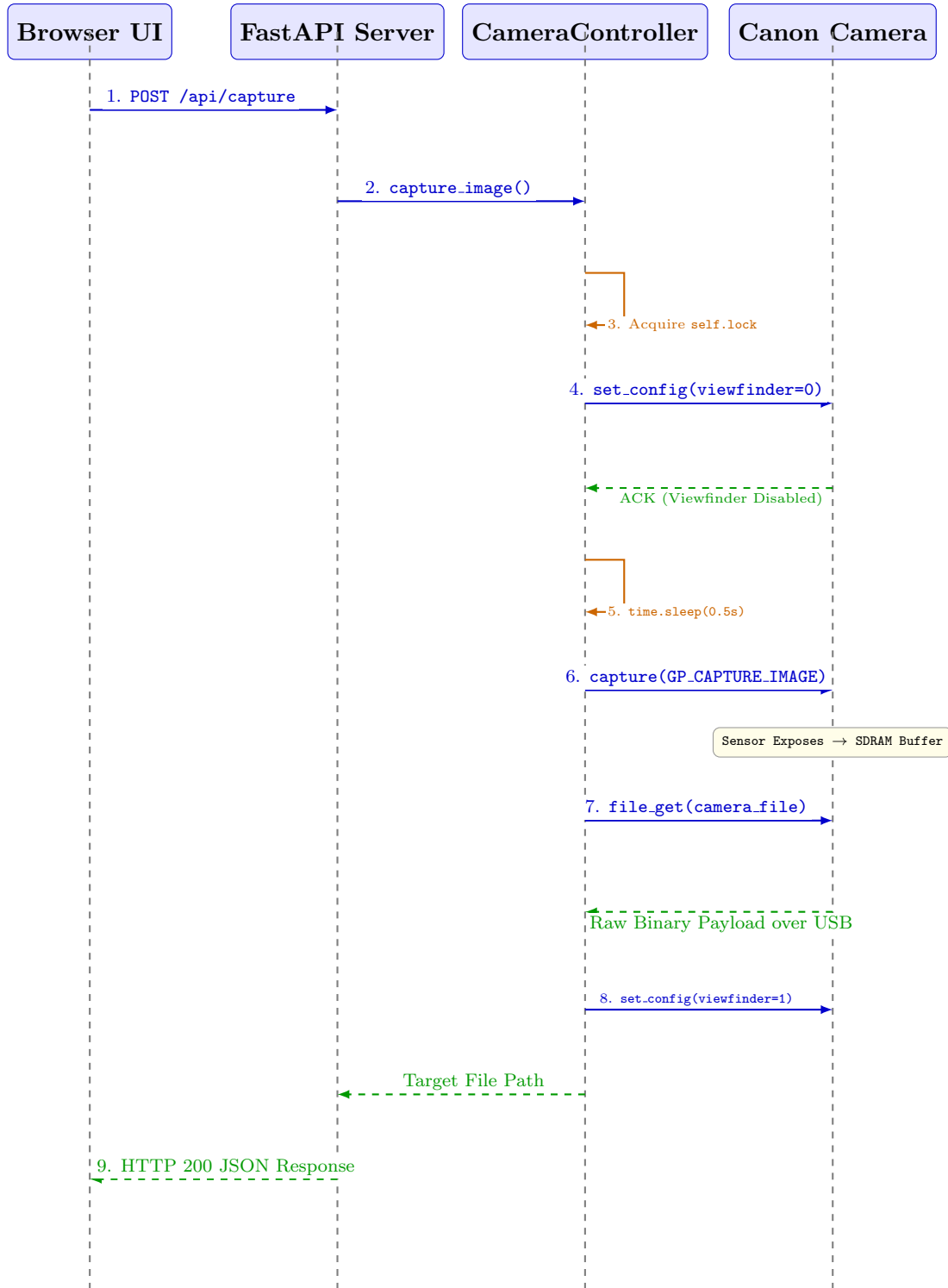


Figure 3: UML Sequence Diagram properly formatted with zero text overlap.

4 Data Extraction and Preview Mechanics

4.1 PTP Capture Target and In-Memory Storage

Upon connection initialization, the controller sets `capturetarget = 1`. This directs the DIGIC X processor to store image data directly in volatile camera SDRAM buffer space

rather than flushing to physical SD card memory, preventing file system locks.

4.2 C-Python Buffer Pointer Retrieval

The binary transfer is executed cleanly without unnecessary file system overhead:

```
1 # Allocate raw C memory file container wrapper
2 camera_file = gp.CameraFile()
3
4 # Fetch binary buffer directly over USB Bulk IN Endpoint
5 self.camera.file_get(file_path.folder, file_path.name,
6                       gp.GP_FILE_TYPE_NORMAL, camera_file, self.
7                       context)
8
9 # Flush binary data buffer to target path
10 camera_file.save(target_path)
```

4.3 Live Preview Frame Extraction

For real-time streaming, the backend continuously pulls preview frames from the viewfinder buffer memory via `capture_preview()`:

```
1 def capture_preview(self):
2     with self.lock:
3         camera_file = gp.CameraFile()
4         self.camera.capture_preview(camera_file, self.context)
5         file_data = camera_file.get_data_and_size()
6         return memoryview(file_data).tobytes()
```

The FastAPI generator packages these frame buffers into an HTTP stream ('multipart/x-mixed-replace; boundary=frame') refreshed at ≈ 20 FPS.

5 Conclusion

The updated UML Use Case and Sequence diagrams fit perfectly within the document layout. By resolving component overlaps and constraining node coordinates within page boundaries, the document provides an easy-to-read illustration of data acquisition flow.